

testing_strategy

Anti-Bluff Testing Strategy: LLMsVerifier Integration into HelixCode

Document Version: 1.0.0
Date: 2026-04-30
Status: Draft — Pending Review
Author: Test Architecture (Anti-Bluff Mandate)
Constitutional Basis: CONST-002, CONST-002a, CONST-005, CONST-006, CONST-017, CONST-020, CONST-021, CONST-025, CONST-035

Table of Contents

- 1. [Anti-Bluff Testing Manifesto](#)
 - 2. [Unit Tests](#)
 - 3. [Contract Tests](#)
 - 4. [Component Tests](#)
 - 5. [Integration Tests](#)
 - 6. [E2E Tests / Challenges](#)
 - 7. [Security Tests](#)
 - 8. [Performance Tests](#)
 - 9. [Coverage Enforcement](#)
 - 10. [Test Infrastructure](#)
 - 11. [Anti-Bluff Verification Checklist Matrix](#)
 - 12. [Appendix A: Makefile Target Definitions](#)
 - 13. [Appendix B: Constitution Cross-Reference](#)
-

1. Anti-Bluff Testing Manifesto

1.1 Definition

Anti-bluff testing is a testing philosophy and methodology that ensures every test PASS genuinely guarantees that the tested feature works for end users. A test is a **bluff** if:

1. It passes when the underlying feature is broken, incomplete, or unusable.
2. It tests code paths rather than observable, user-facing behavior.
3. It uses mocks inappropriately, hiding real failures.
4. It asserts on internal state rather than externally verifiable outcomes.
5. It skips in CI but passes locally under unrealistic conditions.

1.2 The Five Anti-Bluff Principles

#	Principle	Enforcement
AB-001	PASS = Works for Users	Every test must verify that a real user can accomplish a real task. If the user cannot do it, the test must FAIL.
AB-002	No Internal-Only Assertions	Tests must assert on externally observable outcomes (CLI output, API responses, DB state, file contents), not on internal variables or private methods.
AB-003	No Mock Propagation	Mocks are allowed ONLY in unit tests (<code>*_test.go</code> with <code>-short</code>). All other test categories MUST use real dependencies.
AB-004	Prove the Negative	For every positive test (“feature X works”), there MUST be a corresponding negative test (“feature X gracefully fails when Y”).
AB-005	Challenge-Based Validation	Every component MUST have a challenge script that exercises the feature through the exact interface an end user would use.

1.3 Anti-Bluff Enforcement in HelixCode

- **CONST-035:** Every PASS must guarantee quality, completion, usability.

- **CONST-002a:** No mocks above unit tests. Any test above unit that uses a mock causes an immediate build failure.
- **CONST-005:** 100% real data for all non-unit tests.
- **CONST-006:** Every component must have challenge scripts.
- **CONST-020:** Fallback chains must be tested with actual failures, not injected errors.
- **CONST-021:** Makefile must include a `no-mocks-above-unit` target that scans for forbidden mock usage.

1.4 Bluff Detection Checklist (Applied to Every Test)

Before a test is accepted into the suite, it must answer **YES** to all of these:

☐

If I break the feature deliberately, does this test FAIL?

☐

If the feature returns a hardcoded/canned response, does this test FAIL?

☐

If the feature is disconnected from its real dependency, does this test FAIL?

☐

Does this test verify the exact output format a user sees?

☐

Can I run this test in a fresh Docker container and get the same result?

☐

Does this test NOT use `t.Skip()` without a documented SKIP-OK justification?

2. Unit Tests

Scope: Internal package logic, isolated functions, data structures. **Mock**

Policy: Only external HTTP calls may be mocked. Only in `*_test.go` files. Only when `testing.Short()` is true. **Constitutional Basis:**

CONST-002, CONST-002a.

2.1 Mock Policy (Strict)

Rule	Violation Consequence
Mocks only for outbound HTTP/HTTPS calls to external APIs	Build fails
	Build fails

Rule	Violation Consequence
Mocks never for database, cache, filesystem, internal interfaces	
All mock setups wrapped in <code>if testing.Short() { ... } else t.Skip(...)</code>	CI scan rejects
Mock files live only in <code>internal/</code> <code>mocks/</code> or <code>*_test.go</code>	Lint fails
Mock usage logged in test output with <code>t.Log("MOCK-ACTIVE: ...")</code>	Audit trail

2.2 Unit Test File Inventory

All new files go under `helix_code/internal/verifier/` unless noted otherwise.

#	File Path	Purpose	Lines Est.
1	<code>internal/verifier/client_test.go</code>	LLMsVerifier HTTP client (timeouts, retries, auth headers)	~300
2	<code>internal/verifier/config_test.go</code>	Config parsing, validation, defaults, env var binding	~250
3	<code>internal/verifier/scoring_test.go</code>	Score normalization, weight application, cache expiry	~350
4		Model discovery	~300

#	File Path	Purpose	Lines Est.
	internal/verifier/ discovery_test.go	filtering, sorting, deduplication	
5	internal/verifier/ provider_adapter_test.go	Provider adapter initialization, capability mapping	~250
6	internal/verifier/ rate_limit_test.go	Rate limit parsing, cooldown calculation, reset logic	~200
7	internal/verifier/cache_test.go	In-memory cache TTL, eviction, hit/ miss counters	~180
8	internal/verifier/health_test.go	Health status transitions, circuit breaker logic	~250
9	internal/verifier/aliases_test.go	Model alias resolution, fuzzy matching thresholds	~150
10	internal/llm/ verifier_integration_test.go	HelixCode model manager ↔ verifier adapter wiring	~300
11	internal/llm/ verifier_model_manager_test.go	Model manager using verifier data (register, score, select)	~350
12	internal/llm/ verifier_registry_test.go	Cross-provider registry with verifier- supplied models	~250
13			~200

#	File Path	Purpose	Lines Est.
	internal/config/ verifier_config_test.go	HelixCode config loading with verifier section	
14	cmd/cli/verifier_cli_test.go	CLI flag parsing for verifier- related commands	~200
15	internal/services/ llmsverifier_score_adapter_test.go	Score adapter bridge unit tests	~300
16	internal/verifier/ canned_detection_test.go	Anti-bluff: canned response detection logic	~200
17	internal/verifier/fallback_test.go	Fallback model selection when verifier is unavailable	~180
18	internal/verifier/events_test.go	Event publishing, subscription, topic routing	~200
19	internal/verifier/ subscription_detector_test.go	Tier detection (free/paid/ enterprise) logic	~200
20	internal/verifier/ encryption_test.go	SQLite encryption key handling, redaction	~150

Total New Unit Test Files: 20

Estimated Total Lines: ~4,780

2.3 Exact Test Function Signatures and Assertions

File: `internal/verifier/client_test.go`

```
func TestClient_NewClient_WithDefaults(t *testing.T)
func TestClient_NewClient_WithCustomTimeout(t *testing.T)
func TestClient_NewClient_WithCustomHTTPClient(t *testing.T)
func TestClient_GetModels_Success(t *testing.T)           // uses
    mock HTTP only if testing.Short()
func TestClient_GetModels_HTTPError(t
    *testing.T)           // uses mock HTTP only if
    testing.Short()
func TestClient_GetModels_InvalidJSON(t *testing.T)      // uses
    mock HTTP only if testing.Short()
func TestClient_GetModels_Timeout(t *testing.T)          // uses
    mock HTTP only if testing.Short()
func TestClient_GetModelByID_Success(t *testing.T)
func TestClient_GetModelByID_NotFound(t *testing.T)
func TestClient_VerifyModel_Success(t *testing.T)
func TestClient_VerifyModel_ErrorResponse(t *testing.T)
func TestClient_RetryOn5xx(t *testing.T)                 //
    uses mock HTTP only if testing.Short()
func TestClient_RetryExhausted(t *testing.T)             //
    uses mock HTTP only if testing.Short()
func TestClient_AuthHeaderAttached(t *testing.T)
func TestClient_AuthHeaderRedactedInLogs(t *testing.T)
func TestClient_RateLimitHeaderParsing(t *testing.T)
func TestClient_ContextCancellation(t *testing.T)
```

Anti-Bluff Criteria for `client_test.go`: - `TestClient_GetModels_Success`: Must assert that returned `[]ModelInfo` contains at least one model with non-empty ID, Name, and Provider fields. A response with all-zero values or empty strings must cause failure. - `TestClient_AuthHeaderRedactedInLogs`: Must verify that the literal API key string NEVER appears in any log output or error string (search substring).

File: `internal/verifier/config_test.go`

```
func TestConfig_LoadFromFile_YAML(t *testing.T)
func TestConfig_LoadFromFile_JSON(t *testing.T)
func TestConfig_LoadFromFile_TOML(t *testing.T)
func TestConfig_LoadDefaults_WhenFileMissing(t *testing.T)
func TestConfig_EnabledFlag_DefaultTrue(t *testing.T)
func TestConfig_EnabledFlag_OverrideFalse(t *testing.T)
func TestConfig_DatabasePath_Default(t *testing.T)
func TestConfig_DatabaseEncryptionKey_FromEnv(t *testing.T)
func TestConfig_ProviderConfig_OpenAI(t *testing.T)
```

```

func TestConfig_ProviderConfig_Anthropic(t *testing.T)
func TestConfig_ProviderConfig_InvalidProvider(t *testing.T)
func TestConfig_ScoringWeights_SumToOne(t *testing.T)
func TestConfig_ScoringWeights_InvalidSum(t *testing.T)
func TestConfig_HealthCheckInterval(t *testing.T)
func TestConfig_CircuitBreaker_DefaultEnabled(t *testing.T)
func TestConfig_EnvVarSubstitution(t *testing.T)
func TestConfig_EnvVarSubstitution_Missing(t *testing.T)
func TestConfig_Scheduling_ReVerificationInterval(t *testing.T)
func TestConfig_Brotli_EnabledDefault(t *testing.T)
func TestConfig_InvalidYAML_ReturnsError(t *testing.T)

```

Anti-Bluff Criteria for config_test.go: -

TestConfig_EnabledFlag_DefaultTrue: After loading a minimal config file, `cfg.Enabled` must be `true`. An uninitialized boolean (Go zero-value `false`) must cause failure. - TestConfig_ScoringWeights_SumToOne: Must use `math.Abs(sum-1.0) < 0.0001` assertion. If weights are hardcoded incorrectly (e.g., all 0.2 but missing one dimension), the test FAILs. - TestConfig_DatabaseEncryptionKey_FromEnv: Must verify that the encryption key is read from environment and that the struct field is populated — not just that the env var is referenced in the YAML string.

File: internal/verifier/scoring_test.go

```

func TestScoringEngine_New(t *testing.T)
func TestScoringEngine_CalculateScore_AllDimensions(t *testing.T)
func TestScoringEngine_CalculateScore_ZeroWeights(t *testing.T)
func TestScoringEngine_CalculateScore_NormalizeTo10(t *testing.T)
func TestScoringEngine_GetProviderScore_Cached(t *testing.T)
func TestScoringEngine_GetProviderScore_ExpiredCache(t
    *testing.T)
func TestScoringEngine_GetModelScore_MultipleModels(t *testing.T)
func TestScoringEngine_ScoreSuffix_Format(t *testing.T)
func TestScoringEngine_CostEffectiveness_Bonus(t *testing.T)
func TestScoringEngine_CostEffectiveness_Penalty(t *testing.T)
func TestScoringEngine_OpenSourceBonus(t *testing.T)
func TestScoringEngine_BatchScore(t *testing.T)
func TestScoringEngine_HistoryTracking(t *testing.T)
func TestScoringEngine_ScoreRange_0To10(t *testing.T)
func TestScoringEngine_NegativeScore_Clamped(t *testing.T)
func TestScoringEngine_ScoreAbove10_Clamped(t *testing.T)

```

Anti-Bluff Criteria for scoring_test.go: -

TestScoringEngine_CalculateScore_AllDimensions: Must pass a `VerificationResult` with ONLY `code_capability_score` set, and assert the overall score reflects ONLY that dimension's weight. A hardcoded "always 8.5" result must cause failure. - TestScoringEngine_ScoreSuffix_Format: Must

assert regex match `^SC:\d+\. \d+$`. A suffix like “SC:NaN” or “SC:” must cause failure.

File: internal/verifier/discovery_test.go

```
func TestDiscoveryService_New(t *testing.T)
func TestDiscoveryService_DiscoverModels_FilterByProvider(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_FilterByCapability(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_SortByScore(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_Deduplicate(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_MinScoreFilter(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_CodeGeneration(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_StreamingRequired(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_BudgetConstraint(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_NoMatch(t
    *testing.T)
func TestDiscoveryService_ProviderPriority(t *testing.T)
func TestDiscoveryService_CodeVisibilityFilter(t *testing.T)
func TestDiscoveryService_DiversityRequirement(t *testing.T)
```

Anti-Bluff Criteria for discovery_test.go: -

TestDiscoveryService_SelectOptimalModel_CodeGeneration: Must assert that the returned model's Capabilities slice contains "code_generation".

Returning a model without that capability must cause failure. -

TestDiscoveryService_DiscoverModels_Deduplicate: Must feed two models with identical ID but different providers, and assert only one is returned (or both with provider distinction). Returning duplicates without distinction must cause failure.

File: internal/verifier/provider_adapter_test.go

```
func TestProviderAdapter_OpenAI_Init(t *testing.T)
func TestProviderAdapter_Anthropic_Init(t *testing.T)
func TestProviderAdapter_DeepSeek_Init(t *testing.T)
func TestProviderAdapter_Groq_Init(t *testing.T)
func TestProviderAdapter_Mistral_Init(t *testing.T)
func TestProviderAdapter_XAI_Init(t *testing.T)
func TestProviderAdapter_Cohere_Init(t *testing.T)
func TestProviderAdapter_UnsupportedProvider_Error(t *testing.T)
```

```
func TestProviderAdapter_CapabilityMapping(t *testing.T)
func TestProviderAdapter_ModelListConversion(t *testing.T)
```

Anti-Bluff Criteria for provider_adapter_test.go: - Each `_Init` test must verify that the adapter's `GetName()` returns the expected provider name. An empty string or wrong name must cause failure. -

`TestProviderAdapter_CapabilityMapping`: Must map provider-native capability names (e.g., “function_calling”) to unified capability names (e.g., “tools”). A nil or empty mapping must cause failure.

File: internal/verifier/health_test.go

```
func TestHealthService_Check_Healthy(t *testing.T)
func TestHealthService_Check_Degraded(t *testing.T)
func TestHealthService_Check_Unhealthy(t *testing.T)
func TestHealthService_CircuitBreaker_OpenAfterFailures(t
    *testing.T)
func TestHealthService_CircuitBreaker_CloseAfterRecoveries(t
    *testing.T)
func TestHealthService_CircuitBreaker_HalfOpenTimeout(t
    *testing.T)
func TestHealthService_StatusTransitions(t *testing.T)
func TestHealthService_ConcurrentChecks(t *testing.T)
```

Anti-Bluff Criteria for health_test.go: -

`TestHealthService_CircuitBreaker_OpenAfterFailures`: Must call the health check failure_threshold+1 times with an error-returning function, then assert the circuit is Open. A test that asserts after only 1 failure must be rejected. -

`TestHealthService_StatusTransitions`: Must assert the exact transition sequence: unknown → healthy → degraded → unhealthy → offline and verify each transition requires the documented number of events.

File: internal/verifier/canned_detection_test.go

```
func TestIsCannedErrorResponse_MatchesPattern(t *testing.T)
func TestIsCannedErrorResponse_NoMatch(t *testing.T)
func TestIsCannedErrorResponse_EmptyString(t *testing.T)
func TestIsCannedErrorResponse_CaseInsensitive(t *testing.T)
func TestIsSuspiciouslyFastResponse_UnderThreshold(t *testing.T)
func TestIsSuspiciouslyFastResponse_AboveThreshold(t *testing.T)
func TestIsSuspiciouslyFastResponse_ShortContent(t *testing.T)
func TestIsSuspiciouslyFastResponse_LongContent(t *testing.T)
```

Anti-Bluff Criteria for canned_detection_test.go: -

`TestIsCannedErrorResponse_MatchesPattern`: Must test EVERY pattern in `CannedErrorPatterns` at least once via a table-driven test. If a new pattern is added to the source, the test must detect it (or the test must iterate the source

slice). - `TestIsSuspiciouslyFastResponse_UnderThreshold`: Must use `latency = 99 * time.Millisecond` and `contentLen = 30` and assert `true`. Using `latency = 50ms` alone without content check is insufficient.

File: `internal/llm/verifier_model_manager_test.go`

```
func TestModelManager_RegisterVerifierAdapter(t *testing.T)
func TestModelManager_GetAvailableModels_FromVerifier(t
    *testing.T)
func TestModelManager_SelectOptimalModel_UsesVerifierScores(t
    *testing.T)
func TestModelManager_HealthCheck_IncludesVerifierProviders(t
    *testing.T)
func TestModelManager_GetModelsByCapability_VerifierData(t
    *testing.T)
func TestModelManager_Fallback_WhenVerifierOffline(t *testing.T)
func TestModelManager_ModelMetadata_IncludesScoreSuffix(t
    *testing.T)
func TestModelManager_VerifierDisabled_Bypass(t *testing.T)
```

Anti-Bluff Criteria for `verifier_model_manager_test.go`: -

`TestModelManager_GetAvailableModels_FromVerifier`: After registering a verifier adapter, `GetAvailableModels()` must return models with IDs that match the verifier data, NOT the old hardcoded list (`llama-3-8b`, `mistral-7b`, `phi-3-mini`). Returning exactly those 3 models must cause failure. -

`TestModelManager_Fallback_WhenVerifierOffline`: Must simulate verifier API returning 503 for `failure_threshold+1` consecutive calls, then assert the model manager falls back to the next available provider source (e.g., Ollama discovery or hardcoded fallback list). A test that asserts “returns empty” must be rejected.

File: `cmd/cli/verifier_cli_test.go`

```
func TestCLI_ListModelsFlag_Parses(t *testing.T)
func TestCLI_ModelFlag_Parses(t *testing.T)
func TestCLI_VerifierEnabledFlag_Parses(t *testing.T)
func TestCLI_VerifierDisabledFlag_Parses(t *testing.T)
func TestCLI_VerifierConfigFlag_Parses(t *testing.T)
func TestCLI_InteractiveCommand_Models(t *testing.T)
func TestCLI_OutputFormat_JSON(t *testing.T)
func TestCLI_OutputFormat_Table(t *testing.T)
```

Anti-Bluff Criteria for `verifier_cli_test.go`: -

`TestCLI_InteractiveCommand_Models`: Must capture stdout and assert that the output contains model names from the verifier, not the hardcoded 3-model list.

Output containing exactly “Llama 3 8B” / “Mistral 7B” / “Phi-3 Mini” as the only models must cause failure.

3. Contract Tests

Scope: API schema validation, provider API contract verification. **Mock Policy:** NO mocks. Uses real API endpoints with test keys or schema snapshots. **Constitutional Basis:** CONST-005, CONST-002a.

3.1 Contract Test File Inventory

#	File Path	Purpose
1	tests/contract/verifier_api_contract_test.go	Verify LLMsVerifier REST API schema
2	tests/contract/provider_api_contract_test.go	Verify provider API response schemas
3	tests/contract/schema_validation_test.go	JSON schema validation for all API responses
4	tests/contract/model_response_contract_test.go	Verify /api/models response contract
5	tests/contract/verification_response_contract_test.go	Verify /api/models/{id}/verify response contract
6	tests/contract/error_response_contract_test.go	Verify error response format contract

3.2 Exact Test Function Signatures

File: tests/contract/verifier_api_contract_test.go

```
func TestVerifierAPI_HealthEndpoint(t *testing.T)
func TestVerifierAPI_ModelsListEndpoint(t *testing.T)
func TestVerifierAPI_ModelGetEndpoint(t *testing.T)
func TestVerifierAPI_ModelVerifyEndpoint(t *testing.T)
func TestVerifierAPI_ProvidersEndpoint(t *testing.T)
func TestVerifierAPI_ScoreEndpoint(t *testing.T)
func TestVerifierAPI_Headers_CORS(t *testing.T)
func TestVerifierAPI_Headers_ContentType(t *testing.T)
func TestVerifierAPI_Error_404(t *testing.T)
func TestVerifierAPI_Error_401(t *testing.T)
```

Anti-Bluff Criteria: - TestVerifierAPI_ModelsListEndpoint: Must make a REAL HTTP request to the verifier server (running in Docker). Must assert that the response JSON array contains objects with id, name, provider, score, verified fields. Must assert Content-Type: application/json. Must assert HTTP 200. - TestVerifierAPI_Error_401: Must make a request WITHOUT the Authorization header and assert HTTP 401. A test that skips when no auth is configured is a bluff — it must FAIL.

File: tests/contract/provider_api_contract_test.go

```
func TestProviderAPI_OpenAI_ModelsEndpoint(t *testing.T)
func TestProviderAPI_OpenAI_ChatCompletionSchema(t *testing.T)
func TestProviderAPI_Anthropic_MessagesSchema(t *testing.T)
func TestProviderAPI_Anthropic_ModelsEndpoint(t *testing.T)
func TestProviderAPI_DeepSeek_ModelsEndpoint(t *testing.T)
func TestProviderAPI_DeepSeek_ChatCompletionSchema(t *testing.T)
func TestProviderAPI_Groq_ModelsEndpoint(t *testing.T)
func TestProviderAPI_XAI_ModelsEndpoint(t *testing.T)
func TestProviderAPI_Mistral_ModelsEndpoint(t *testing.T)
```

Anti-Bluff Criteria: - Each TestProviderAPI_*_ModelsEndpoint: Must make a REAL HTTP request to the provider's actual models endpoint using a test API key from environment. Must assert the response is valid JSON with at least one model object. Must assert the model object has id and object fields. If the provider API is unreachable or returns an error, the test must FAIL (with a SKIP-OK only if the env var is missing, documented). - Each TestProviderAPI_*_ChatCompletionSchema: Must make a REAL chat completion request with a minimal prompt (e.g., "Say 'hello'"), assert 200 OK, assert the response has .choices[0].message.content containing non-empty text.

File: tests/contract/schema_validation_test.go

```
func TestSchema_ModelInfo(t *testing.T)
func TestSchema_VerificationResult(t *testing.T)
func TestSchema_ProviderInfo(t *testing.T)
func TestSchema_ScoreDetails(t *testing.T)
func TestSchema_ErrorResponse(t *testing.T)
func TestSchema_Config(t *testing.T)
```

Anti-Bluff Criteria: - TestSchema_ModelInfo: Must validate a ModelInfo struct against the actual JSON returned from the verifier /api/models endpoint. The struct must have json:"..." tags for every field in the response. Missing fields must cause failure. - TestSchema_VerificationResult: Must validate that the verification result struct has fields for ALL dimensions: code_capability_score, responsiveness_score, reliability_score,

feature_richness_score,value_proposition_score. Missing any dimension field must cause failure.

3.3 Schema Snapshot Files

Stored in tests/contract/snapshots/:

File	Purpose
verifier_models_list.json	Expected JSON schema for /api/models
verifier_model_get.json	Expected JSON schema for /api/models/{id}
verifier_verify_result.json	Expected JSON schema for /api/models/{id}/verify
verifier_error.json	Expected JSON schema for error responses
provider_openai_models.json	Expected schema for OpenAI /v1/models
provider_anthropic_messages.json	Expected schema for Anthropic /v1/messages
provider_deepseek_chat.json	Expected schema for DeepSeek /chat/completions

These snapshots are generated by the contract tests on the first run and then used for structural validation. If the real API changes its schema, the snapshot mismatch causes a test failure.

4. Component Tests

Scope: Real subsystems wired together, no external mocks. **Mock**

Policy: NO mocks. All subsystems are real instances (in-memory SQLite, real cache, real config structs). **Constitutional Basis:** CONST-005, CONST-006.

4.1 Component Test File Inventory

#	File Path	Purpose
1	tests/component/ verifier_client_cache_component_test.go	Verifier client + in-memory cache + config

#	File Path	Purpose
2	tests/component/ model_manager_verifier_component_test.go	Model manager + verifier adapter + scoring
3	tests/component/ cli_output_formatter_component_test.go	CLI formatter + real model data structures
4	tests/component/ discovery_scoring_component_test.go	Discovery service + scoring service + health service
5	tests/component/ startup_pipeline_component_test.go	Startup verifier (phases 1-5) with real subsystems
6	tests/component/ event_bus_verifier_component_test.go	Event publisher + verifier events + subscriber

4.2 Exact Test Function Signatures

File: tests/component/verifier_client_cache_component_test.go

```

func TestClientCache_CacheHit_AvoidsHTTPCall(t *testing.T)
func TestClientCache_CacheMiss_MakesHTTPCall(t *testing.T)
func TestClientCache_TTL_Expires(t *testing.T)
func TestClientCache_Eviction_MaxSize(t *testing.T)
func TestClientCache_CacheDisabled_Bypasses(t *testing.T)
func TestClientCache_ConcurrentAccess(t *testing.T)

```

Anti-Bluff Criteria: - TestClientCache_CacheHit_AvoidsHTTPCall: Must use a real HTTP server (httptest) with a request counter. On the second call with the same key, the request counter must NOT increment. If the counter increments, the cache is not working — test FAILs. - TestClientCache_TTL_Expires: Must set TTL to 1 second, wait 2 seconds, then assert the cache returns a miss. A test that asserts “still valid after 2s” must FAIL.

File: tests/component/model_manager_verifier_component_test.go

```

func TestModelManager_RegisterVerifierProvider(t *testing.T)
func TestModelManager_GetModels_ReturnsVerifierModels(t
    *testing.T)
func TestModelManager_SelectModel_UsesVerifierScores(t
    *testing.T)

```

```

func TestModelManager_HealthCheck_ReflectsVerifierStatus(t
    *testing.T)
func TestModelManager_Fallback_ToLocalProvider(t *testing.T)
func TestModelManager_ScoreSuffix_Display(t *testing.T)
func TestModelManager_ConcurrentRegisterAndSelect(t *testing.T)

```

Anti-Bluff Criteria: -

TestModelManager_GetModels_ReturnsVerifierModels: Must register a verifier provider that returns 5 real models from a test SQLite DB, then call GetAvailableModels() and assert the returned count is 5 and each ID matches the DB. Returning 3 hardcoded models must cause failure. -

TestModelManager_SelectModel_UsesVerifierScores: Must insert two models into the verifier DB — one with score 9.0, one with score 5.0. Call SelectOptimalModel() and assert the 9.0 model is selected. If the lower-scored model is selected, the scoring integration is broken — test FAILs.

File: tests/component/cli_output_formatter_component_test.go

```

func TestCLIFormatter_TableOutput_ContainsModelNames(t
    *testing.T)
func TestCLIFormatter_TableOutput_ContainsScoreSuffix(t
    *testing.T)
func TestCLIFormatter_TableOutput_ContainsVerificationBadge(t
    *testing.T)
func TestCLIFormatter_JSONOutput_ValidJSON(t *testing.T)
func TestCLIFormatter_JSONOutput_ContainsAllFields(t *testing.T)
func TestCLIFormatter_EmptyModels_HandlesGracefully(t *testing.T)

```

Anti-Bluff Criteria: -

TestCLIFormatter_TableOutput_ContainsScoreSuffix: Must pass a model with ScoreSuffix = "SC:8.5" to the formatter and assert the output string contains "SC:8.5". If the formatter ignores the suffix, test FAILs. -

TestCLIFormatter_JSONOutput_ContainsAllFields: Must assert that JSON output contains ALL of: id, name, provider, score, verified, latency, context_window. Missing any field must cause failure.

File: tests/component/startup_pipeline_component_test.go

```

func TestStartupPipeline_Phase1_DiscoverProviders(t *testing.T)
func TestStartupPipeline_Phase2_VerifyProviders(t *testing.T)
func TestStartupPipeline_Phase2_5_DetectSubscriptions(t
    *testing.T)
func TestStartupPipeline_Phase3_ScoreProviders(t *testing.T)
func TestStartupPipeline_Phase4_RankProviders(t *testing.T)
func TestStartupPipeline_Phase5_SelectDebateTeam(t *testing.T)

```

```
func TestStartupPipeline_AllPhases_EndToEnd(t *testing.T)
func TestStartupPipeline_ProviderWithFaultyKey_Deprioritized(t
    *testing.T)
```

Anti-Bluff Criteria: - TestStartupPipeline_Phase1_DiscoverProviders:

Must assert that SupportedProviders map is iterated and providers with available env vars are discovered. A test that asserts “3 providers discovered” without checking which ones is a bluff. -

TestStartupPipeline_AllPhases_EndToEnd: Must run all 5 phases with a test SQLite DB containing 2 providers and 4 models, and assert the final DebateTeamResult contains at least 1 model. An empty debate team must cause failure.

5. Integration Tests

Scope: Full application with real dependencies (PostgreSQL, Redis, SQLite, real provider APIs). **Mock Policy:** NO mocks whatsoever. Real API keys from environment. Real databases from Docker.

Constitutional Basis: CONST-005, CONST-020.

5.1 Integration Test File Inventory

#	File Path	Purpose
1	tests/integration/helixcode_verifier_sqlite_test.go	HelixCode + LLMsVerifier SQLite DB
2	tests/integration/helixcode_provider_api_test.go	HelixCode + real provider APIs (with test keys)
3	tests/integration/helixcode_redis_cache_test.go	HelixCode + Redis cache for verifier data
4	tests/integration/helixcode_postgres_test.go	HelixCode + PostgreSQL with verifier tables
5	tests/integration/helixcode_full_stack_test.go	Server + DB + Cache + Verifier + Provider
6	tests/integration/helixcode_verifier_events_test.go	Event bus + verifier + WebSocket
7		

#	File Path	Purpose
	tests/integration/helixcode_fallback_chain_test.go	Provider fallback with real failures
8	tests/integration/helixcode_verifier_config_reload_test.go	Config hot-reload with verifier changes

5.2 Exact Test Function Signatures

File: tests/integration/helixcode_verifier_sqlite_test.go

```

func TestHelixCodeVerifierSQLite_DBCConnection(t *testing.T)
func TestHelixCodeVerifierSQLite_ModelCRUD(t *testing.T)
func TestHelixCodeVerifierSQLite_VerificationResultPersisted(t
    *testing.T)
func TestHelixCodeVerifierSQLite_ProviderMetadata(t *testing.T)
func TestHelixCodeVerifierSQLite_RateLimits(t *testing.T)
func TestHelixCodeVerifierSQLite_PricingData(t *testing.T)
func TestHelixCodeVerifierSQLite_ConcurrentAccess(t *testing.T)
func TestHelixCodeVerifierSQLite_EncryptionEnabled(t *testing.T)

```

Anti-Bluff Criteria: - TestHelixCodeVerifierSQLite_ModelCRUD: Must CREATE a model, READ it back, UPDATE its score, DELETE it, then assert the model is gone. Each step must verify the exact DB row via SELECT. A test that only creates and reads is a bluff. -

TestHelixCodeVerifierSQLite_VerificationResultPersisted: Must run a real verification through the verifier client, then query the SQLite verification_results table and assert a row exists with matching model_id and non-zero overall_score. If the table is empty, test FAILS. -

TestHelixCodeVerifierSQLite_EncryptionEnabled: Must start the verifier with encryption_enabled: true, insert a model, then attempt to read the raw SQLite file bytes and assert the content is NOT plaintext (search for model name string in file bytes). Finding the plaintext model name in the file must cause failure.

File: tests/integration/helixcode_provider_api_test.go

```

func TestHelixCodeProviderAPI_OpenAI_RealModelList(t *testing.T)
func TestHelixCodeProviderAPI_OpenAI_RealChatCompletion(t
    *testing.T)
func TestHelixCodeProviderAPI_Anthropic_RealMessages(t
    *testing.T)
func TestHelixCodeProviderAPI_DeepSeek_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_Groq_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_XAI_RealChat(t *testing.T)

```

```

func TestHelixCodeProviderAPI_Mistral_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_InvalidKey_PropagatesError(t
    *testing.T)
func TestHelixCodeProviderAPI_RateLimit_PropagatesError(t
    *testing.T)

```

Anti-Bluff Criteria: - TestHelixCodeProviderAPI_OpenAI_RealModelList: Must call the real OpenAI /v1/models endpoint with a test API key, parse the response, and assert at least 10 models are returned. If 0 models are returned, test FAILs. If the response is mocked, test FAILs. -

TestHelixCodeProviderAPI_OpenAI_RealChatCompletion: Must send a real chat completion request with prompt "Say exactly 'ANTIBLUFF-OK'", and assert the response content contains "ANTIBLUFF-OK". A hardcoded or simulated response must cause failure. -

TestHelixCodeProviderAPI_InvalidKey_PropagatesError: Must use an intentionally invalid API key (e.g., sk-invalid-test-key-12345), call the provider, and assert the error message is propagated to the user-visible layer. Swallowing the error or returning a generic success must cause failure.

File: tests/integration/helixcode_redis_cache_test.go

```

func TestHelixCodeRedisCache_ModelListCached(t *testing.T)
func TestHelixCodeRedisCache_ModelListCacheExpiry(t *testing.T)
func TestHelixCodeRedisCache_VerificationResultCached(t
    *testing.T)
func TestHelixCodeRedisCache_ScoreCached(t *testing.T)
func TestHelixCodeRedisCache_RedisDown_GracefulFallback(t
    *testing.T)
func TestHelixCodeRedisCache_CacheInvalidation(t *testing.T)

```

Anti-Bluff Criteria: - TestHelixCodeRedisCache_ModelListCached: Must call GetAvailableModels() twice. The second call must be served from Redis (verify via redis-cli MONITOR or INFO stats). If the second call hits the verifier API again, the cache is not working — test FAILs. -

TestHelixCodeRedisCache_RedisDown_GracefulFallback: Must stop the Redis container mid-test, then call GetAvailableModels() and assert the result is still returned (from verifier directly or from in-memory fallback). Returning an error to the user when Redis is down is a bluff — the system must still work.

File: tests/integration/helixcode_postgres_test.go

```

func TestHelixCodePostgres_VerifierSchema_Migrated(t *testing.T)
func TestHelixCodePostgres_VerifierData_SyncedFromSQLite(t
    *testing.T)

```

```
func TestHelixCodePostgres_VerifierQuery_Performance(t
    *testing.T)
func TestHelixCodePostgres_VerifierTransaction_Rollback(t
    *testing.T)
```

Anti-Bluff Criteria: - TestHelixCodePostgres_VerifierSchema_Migrated: Must query `information_schema.tables` and assert tables for verifier data exist. If the schema migration was skipped, test FAILs.

File: `tests/integration/helixcode_fallback_chain_test.go`

```
func TestFallbackChain_PrimaryFails_SecondaryUsed(t *testing.T)
func TestFallbackChain_AllFail_ErrorReturned(t *testing.T)
func TestFallbackChain_Recovery_PrimaryReused(t *testing.T)
func TestFallbackChain_RealFailure_NoMock(t *testing.T)
```

Anti-Bluff Criteria: - TestFallbackChain_RealFailure_NoMock: Must configure a primary provider with a wrong endpoint (e.g., `http://localhost:59999` where nothing listens), and a secondary provider with a real endpoint. Assert the secondary is used. If both are mocked, test FAILs.

File: `tests/integration/helixcode_full_stack_test.go`

```
func TestFullStack_ServerStarts_WithVerifier(t *testing.T)
func TestFullStack_APIModels_ReturnsVerifierData(t *testing.T)
func TestFullStack_CLIListModel_ReturnsVerifierData(t
    *testing.T)
func TestFullStack_WebSocket_EmitsVerificationEvents(t
    *testing.T)
func TestFullStack_HealthCheck_IncludesVerifier(t *testing.T)
func TestFullStack_Generate_WithVerifiedModel(t *testing.T)
```

Anti-Bluff Criteria: - TestFullStack_APIModels_ReturnsVerifierData: Must start the full server stack (PostgreSQL, Redis, LLMsVerifier server), make a GET `/api/v1/models` request, and assert the response contains model data from the verifier SQLite DB (not hardcoded). Hardcoded model IDs must cause failure. - TestFullStack_Generate_WithVerifiedModel: Must send a POST `/api/v1/tasks` with a verified model ID, wait for the task to complete, and assert the response contains actual generated text (not a canned “Generated response for…” string). The canned prefix must cause failure.

6. E2E Tests / Challenges

Scope: Complete user workflows through the exact interfaces users interact with. **Mock Policy:** NO mocks. Real CLI binary, real server, real API keys. **Constitutional Basis:** CONST-006, CONST-035.

6.1 Challenge Script Inventory

All challenge scripts live in `challenges/scripts/` and are shell scripts (bash) with strict error handling (`set -euo pipefail`).

#	Script File	Purpose	Lines Est.
1	<code>challenges/scripts/verifier_model_list_challenge.sh</code>	List models and verify they're from verifier, not hardcoded	~180
2	<code>challenges/scripts/verifier_model_select_challenge.sh</code>	Select a model, verify it passes validation, generate code	~220
3	<code>challenges/scripts/verifier_disable_fallback_challenge.sh</code>	Disable verifier, verify fallback to old behavior	~160
4	<code>challenges/scripts/verifier_api_key_provision_challenge.sh</code>	Verify all provider API keys are provisioned through config	~150
5	<code>challenges/scripts/verifier_rate_limit_display_challenge.sh</code>	Verify rate-limited models are marked disabled	~180

#	Script File	Purpose	Lines Est.
		with clear notes	
6	challenges/scripts/ verifier_realtime_update_challenge.sh	Verify real-time updates reflect in model list within N seconds	~200
7	challenges/scripts/ verifier_mcp_lsp_acp_challenge.sh	Verify MCP/LSP/ACP/Embedding integration works end-to-end	~250
8	challenges/scripts/ verifier_cross_platform_cli_challenge.sh	Cross-platform CLI output verification	~180
9	challenges/scripts/ verifier_startup_pipeline_challenge.sh	Verify 5-phase startup pipeline completes with real providers	~220
10	challenges/scripts/ verifier_canned_detection_challenge.sh	Verify canned response detection marks models unverified	~170
11	challenges/scripts/ verifier_security_redaction_challenge.sh	Verify API keys never appear in logs, stdout, or error messages	~160

#	Script File	Purpose	Lines Est.
12	challenges/scripts/verifier_scoring_accuracy_challenge.sh	Verify scoring reflects real verification results, not hardcoded 8.5	~190

6.2 Challenge Script Templates

Challenge 1: verifier_model_list_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# CONST-035: End-User Usability Mandate
# ANTI-BLUFF: This challenge proves that "model listing works" =
# the CLI shows real models from verifier DB, not hardcoded
# list.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
VERIFIER_DB="${PROJECT_ROOT}/test_data/verifier.db"

# --- Setup ---
echo "[CHALLENGE] verifier_model_list_challenge: START"

# Ensure verifier DB has test models
sqlite3 "${VERIFIER_DB}" <<EOF
DELETE FROM models;
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (1, 'test-gpt-4o', 'GPT-4o', 'OpenAI GPT-4o model', 9.2,
        'verified');
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (2, 'test-claude-sonnet', 'Claude Sonnet 4', 'Anthropic
        Claude Sonnet', 8.8, 'verified');
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (3, 'test-deepseek-chat', 'DeepSeek Chat', 'DeepSeek V3',
        8.5, 'verified');
EOF
```

```

# --- Action: List models via CLI ---
OUTPUT_FILE="/tmp/verifier_model_list_output.txt"
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Must contain verifier models
if ! grep -q "GPT-4o" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verifier model 'GPT-4o'"
    exit 1
fi

if ! grep -q "Claude Sonnet 4" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verifier model 'Claude
    Sonnet 4'"
    exit 1
fi

if ! grep -q "DeepSeek Chat" "${OUTPUT_FILE}"; then
    echo
    "[FAIL] Output does not contain verifier model 'DeepSeek
    Chat'"
    exit 1
fi

# ANTI-BLUFF 2: Must NOT contain ONLY the old hardcoded 3-model
    list
HARDCODED_COUNT=$(grep -c -E "Llama 3 8B|Mistral 7B|Phi-3 Mini"
    "${OUTPUT_FILE}" || true)
TOTAL_MODEL_COUNT=$(grep -c -E "^[a-zA-Z]" "${OUTPUT_FILE}" ||
    true)

if [[ "${HARDCODED_COUNT}" -ge 3 && "${TOTAL_MODEL_COUNT}" -le
    4 ]]; then
    echo
    "[FAIL] Output appears to contain only the old hardcoded
    3-model list"
    exit 1
fi

# ANTI-BLUFF 3: Must contain score suffix for verified models
if ! grep -q "SC:" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain score suffix (SC:X.X)"
    exit 1
fi

# ANTI-BLUFF 4: Must contain verification badge

```

```

if ! grep -q -E "(✓|verified|Verified)" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verification indicator"
    exit 1
fi

```

```

echo "[CHALLENGE] verifier_model_list_challenge: PASS"

```

Challenge 2: verifier_model_select_challenge.sh

```

#!/usr/bin/env bash

```

```

set -euo pipefail

```

```

# ANTI-BLUFF: This challenge proves that "model selection + code
#           generation works" =
#   selecting a verified model actually produces real code
#           output.

```

```

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"

```

```

# --- Setup ---

```

```

echo "[CHALLENGE] verifier_model_select_challenge: START"

```

```

# --- Action: Generate code with a verified model ---

```

```

OUTPUT_FILE="/tmp/verifier_generate_output.txt"
"${CLI_BIN}" \
    --model "test-gpt-4o" \
    --prompt "Write a Go function named AntiBluffVerify that
    returns true" \
    > "${OUTPUT_FILE}" 2>&1 || true

```

```

# --- Assertions ---

```

```

# ANTI-BLUFF 1: Output must contain real Go code, not a simulated
#           placeholder

```

```

if grep -q "Generated response for:" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output contains simulated placeholder text"
    exit 1
fi

```

```

# ANTI-BLUFF 2: Output must contain the requested function name

```

```

if ! grep -q "AntiBluffVerify" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain requested function name
    'AntiBluffVerify'"
    exit 1
fi

```

```

# ANTI-BLUFF 3: Output must contain 'func' keyword (it's Go code)
if ! grep -q "func" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain Go 'func' keyword"
    exit 1
fi

# ANTI-BLUFF 4: Output must contain 'return' statement
if ! grep -q "return" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain 'return' statement"
    exit 1
fi

# ANTI-BLUFF 5: Output must NOT contain "TODO" or "coming soon"
if grep -qiE "(TODO|coming soon|not implemented|placeholder)" "$
    {OUTPUT_FILE}"; then
    echo "[FAIL] Output contains incomplete placeholder text"
    exit 1
fi

echo "[CHALLENGE] verifier_model_select_challenge: PASS"

```

Challenge 3: verifier_disable_fallback_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "verifier disable +
#             fallback works" =
#             when verifier is disabled, the system falls back to previous
#             behavior
#             (Ollama discovery, hardcoded list, or local provider) and
#             still functions.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
CONFIG_FILE="/tmp/verifier_disabled_config.yaml"

# --- Setup: Create config with verifier disabled ---
cat > "${CONFIG_FILE}" <<EOF
version: "1.0.0"
llm:
  default_provider: "local"
  default_model: "llama-3.2-3b"
verifier:
  enabled: false
providers:
  ollama:

```

```
    enabled: true
    base_url: "http://localhost:11434"
EOF
```

```
echo "[CHALLENGE] verifier_disable_fallback_challenge: START"

# --- Action: List models with verifier disabled ---
OUTPUT_FILE="/tmp/verifier_disabled_output.txt"
HELIX_CONFIG="${CONFIG_FILE}" "${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Must NOT crash or return error
if grep -qiE "(error|fatal|panic|exception)" "${OUTPUT_FILE}";
then
    echo "[FAIL] CLI returned error when verifier is disabled"
    exit 1
fi

# ANTI-BLUFF 2: Must return SOME models (from fallback source)
MODEL_COUNT=$(grep -c -E "^[a-zA-Z0-9]" "${OUTPUT_FILE}" || true)
if [[ "${MODEL_COUNT}" -lt 1 ]]; then
    echo "[FAIL] No models returned when verifier is disabled
(fallback broken)"
    exit 1
fi

# ANTI-BLUFF 3: Must NOT reference verifier-specific models if
none are available
# (This is acceptable – the test only requires that it doesn't
crash)

echo "[CHALLENGE] verifier_disable_fallback_challenge: PASS"
```

Challenge 4: verifier_api_key_provision_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "API key provisioning
works" =
# all provider API keys are read from config/env, never
hardcoded,
# and are actually used in provider initialization.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
```

```

CONFIG_FILE="${PROJECT_ROOT}/helix_code/config/config.yaml"
ENV_FILE="${PROJECT_ROOT}/.env"

echo "[CHALLENGE] verifier_api_key_provision_challenge: START"

# --- Assertions ---

# ANTI-BLUFF 1: Config file must reference env vars for API keys,
# not literal values
if grep -qE "api_key:\s*[A-Za-z0-9_-]{20,}" "${CONFIG_FILE}" 2>/
dev/null; then
    echo "[FAIL] Config file contains literal API key (not env
var reference)"
    exit 1
fi

# ANTI-BLUFF 2: .env.example must list all provider API keys
REQUIRED_KEYS=(
    "OPENAI_API_KEY"
    "ANTHROPIC_API_KEY"
    "DEEPSEEK_API_KEY"
    "GROQ_API_KEY"
    "MISTRAL_API_KEY"
    "XAI_API_KEY"
    "TOGETHER_API_KEY"
    "OPENROUTER_API_KEY"
)

for KEY in "${REQUIRED_KEYS[@]}; do
    if ! grep -q "${KEY}" "${ENV_FILE}" 2>/dev/null; then
        echo "[FAIL] .env does not document required key: ${KEY}"
        exit 1
    fi
done

# ANTI-BLUFF 3: Verifier config must use env var substitution
# pattern
VERIFIER_CONFIG="${PROJECT_ROOT}/configs/verifier.yaml"
if grep -qE "api_key:\s*[^$\\"]" "${VERIFIER_CONFIG}" 2>/dev/
null; then
    echo "[FAIL] Verifier config contains hardcoded API key"
    exit 1
fi

# ANTI-BLUFF 4: At least one test key must be present for
# integration tests
TEST_KEY_COUNT=0
for KEY in "${REQUIRED_KEYS[@]}; do

```

```

ENV_VAL="${!KEY:-}"
if [[ -n "${ENV_VAL}" && "${ENV_VAL}" != "your-*" ]]; then
    ((TEST_KEY_COUNT++)) || true
fi
done

if [[ "${TEST_KEY_COUNT}" -lt 1 ]]; then
    echo "[WARN] No test API keys found in environment. Skipping
        live provider tests."
    # This is SKIP-OK per CONST-035 – documented in AGENTS.md
    exit 0
fi

echo "[CHALLENGE] verifier_api_key_provision_challenge: PASS ($
    {TEST_KEY_COUNT} keys provisioned)"

```

Challenge 5: verifier_rate_limit_display_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "rate limiting display
#           works" =
#           deliberately exhausting a provider quota causes the model to
#           be marked
#           with a cooldown indicator within the refresh interval.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
REFRESH_INTERVAL=30 # seconds

echo "[CHALLENGE] verifier_rate_limit_display_challenge: START"

# --- Setup: Insert a model with rate limit into verifier DB ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
UPDATE models SET verification_status = 'rate_limited' WHERE
    model_id = 'test-gpt-4o';
INSERT OR REPLACE INTO limits (model_id, limit_type, limit_value,
    current_usage, reset_period)
VALUES ((SELECT id FROM models WHERE model_id = 'test-gpt-4o'),
    'requests_per_minute', 3, 3, '1m');
EOF

# Wait for refresh interval
sleep "${REFRESH_INTERVAL}"

# --- Action: List models ---

```

```

OUTPUT_FILE="/tmp/verifier_rate_limit_output.txt"
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Rate-limited model must be marked (disabled,
#               cooldown, or similar)
if ! grep -q -iE "(rate.?limited|cooldown|disabled|unavailable|
exhausted)" "${OUTPUT_FILE}"; then
    echo "[FAIL] Rate-limited model not marked in output"
    exit 1
fi

# ANTI-BLUFF 2: Must NOT allow selecting the rate-limited model
#               without warning
if grep -q "test-gpt-4o.*available" "${OUTPUT_FILE}"; then
    echo "[FAIL] Rate-limited model still shown as 'available'"
    exit 1
fi

echo "[CHALLENGE] verifier_rate_limit_display_challenge: PASS"

```

Challenge 6: verifier_realtime_update_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "real-time updates work"
#             =
#             modifying the verifier DB causes the CLI model list to
#             reflect changes
#             within N seconds (the configured refresh interval).

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
REFRESH_INTERVAL=30
MAX_WAIT=$((REFRESH_INTERVAL + 10))

echo "[CHALLENGE] verifier_realtime_update_challenge: START"

# --- Step 1: Baseline list ---
BASELINE_FILE="/tmp/verifier_rt_baseline.txt"
"${CLI_BIN}" --list-models > "${BASELINE_FILE}" 2>&1 || true

# --- Step 2: Add a new model to verifier DB ---
NEW_MODEL_NAME="RealtimeTestModel-$(date +%s)"
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF

```

```

INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (1, 'test-realtime-model', '${NEW_MODEL_NAME}', 'Inserted
        for realtime test', 7.5, 'verified');
EOF

```

```

# --- Step 3: Poll until model appears or timeout ---
START_TIME=$(date +%s)
FOUND=0
while true; do
    CURRENT_FILE="/tmp/verifier_rt_current.txt"
    "${CLI_BIN}" --list-models > "${CURRENT_FILE}" 2>&1 || true

    if grep -q "${NEW_MODEL_NAME}" "${CURRENT_FILE}"; then
        FOUND=1
        break
    fi

    ELAPSED=$((($(date +%s) - START_TIME))
    if [[ "${ELAPSED}" -ge "${MAX_WAIT}" ]]; then
        break
    fi

    sleep 2
done

```

```

# --- Assertions ---

if [[ "${FOUND}" -eq 0 ]]; then
    echo "[FAIL] New model did not appear in CLI output within $
        {MAX_WAIT}s"
    exit 1
fi

ELAPSED=$((($(date +%s) - START_TIME))
echo "[CHALLENGE] verifier_realtime_update_challenge: PASS
        (reflected in ${ELAPSED}s)"

```

Challenge 7: verifier_mcp_lsp_acp_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "MCP/LSP/ACP/Embedding
    integration works" =
# the verifier data is accessible through all protocol
    interfaces.

```

```

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_URL="http://localhost:8080"

echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: START"

# --- MCP: Model Context Protocol test ---
MCP_OUTPUT="/tmp/verifier_mcp_output.txt"
curl -s "${SERVER_URL}/api/v1/mcp/models" > "${MCP_OUTPUT}" 2>&1 || true

if ! grep -q "test-gpt-4o" "${MCP_OUTPUT}"; then
    echo "[FAIL] MCP endpoint does not return verifier models"
    exit 1
fi

# --- LSP: Language Server Protocol test (if applicable) ---
LSP_OUTPUT="/tmp/verifier_lsp_output.txt"
curl -s "${SERVER_URL}/api/v1/lsp/completion" \
    -H "Content-Type: application/json" \
    -d '{"model": "test-gpt-4o", "prompt": "func AntiBluff"}' > "${LSP_OUTPUT}" 2>&1 || true

if ! grep -q "AntiBluff" "${LSP_OUTPUT}"; then
    echo "[FAIL] LSP endpoint does not return completions with verifier model"
    exit 1
fi

# --- ACP: Agent Communication Protocol test ---
ACP_OUTPUT="/tmp/verifier_acp_output.txt"
curl -s "${SERVER_URL}/api/v1/acp/agents/discover" > "${ACP_OUTPUT}" 2>&1 || true

if ! grep -q "verifier" "${ACP_OUTPUT}"; then
    echo "[FAIL] ACP endpoint does not reference verifier"
    exit 1
fi

# --- Embedding: Verify embedding model from verifier ---
EMBED_OUTPUT="/tmp/verifier_embed_output.txt"
curl -s "${SERVER_URL}/api/v1/embeddings" \
    -H "Content-Type: application/json" \
    -d '{"model": "text-embedding-3-small", "input": "anti-bluff test"}' > "${EMBED_OUTPUT}" 2>&1 || true

if ! grep -q "embedding" "${EMBED_OUTPUT}"; then
    echo "[FAIL] Embedding endpoint does not return embeddings"

```

```
        exit 1
    fi

    echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: PASS"
```

Challenge 8: verifier_cross_platform_cli_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "CLI output is correct
#             on all platforms" =
#             the same command produces structurally identical output on
#             Linux, macOS, Windows.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"

echo "[CHALLENGE] verifier_cross_platform_cli_challenge: START"

PLATFORM=$(uname -s)
ARCH=$(uname -m)
OUTPUT_FILE="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.txt"
JSON_OUTPUT="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.json"

# --- Action: Table output ---
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Action: JSON output ---
"${CLI_BIN}" --list-models --format json > "${JSON_OUTPUT}" 2>&1
|| true

# --- Assertions ---

# ANTI-BLUFF 1: Table output must not contain platform-specific
#               artifacts
if grep -q $'\r' "${OUTPUT_FILE}"; then
    echo "[FAIL] Table output contains CRLF line endings (Windows
    artifact on non-Windows)"
    exit 1
fi

# ANTI-BLUFF 2: JSON output must be valid JSON on all platforms
if ! python3 -m json.tool "${JSON_OUTPUT}" > /dev/null 2>&1; then
```

```

    echo "[FAIL] JSON output is not valid JSON on ${PLATFORM}"
    exit 1
fi

# ANTI-BLUFF 3: JSON must contain the same top-level keys
# regardless of platform
EXPECTED_KEYS='["id","name","provider","score","verified"]'
ACTUAL_KEYS=$(python3 -c "
import json, sys
data = json.load(open('${JSON_OUTPUT}'))
if isinstance(data, list) and len(data) > 0:
    print(json.dumps(sorted(data[0].keys())))
else:
    print('[]')
")

for KEY in "id" "name" "provider" "score" "verified"; do
    if ! echo "${ACTUAL_KEYS}" | grep -q "\"${KEY}\""; then
        echo "[FAIL] JSON missing required key '${KEY}' on $
        {PLATFORM}"
        exit 1
    fi
done

# ANTI-BLUFF 4: Model count must be consistent (within tolerance
# for platform-specific providers)
MODEL_COUNT=$(python3 -c "
import json
data = json.load(open('${JSON_OUTPUT}'))
print(len(data)) if isinstance(data, list) else print(0)
")

if [[ "${MODEL_COUNT}" -lt 1 ]]; then
    echo "[FAIL] No models in JSON output on ${PLATFORM}"
    exit 1
fi

echo "[CHALLENGE] verifier_cross_platform_cli_challenge: PASS ($
    {PLATFORM} ${ARCH}, ${MODEL_COUNT} models)"

```

Challenge 9: verifier_startup_pipeline_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "startup pipeline works"
#
# the 5-phase startup completes, discovers real providers, and

```

```

#   selects a non-empty debate team.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_BIN="${PROJECT_ROOT}/helix_code/bin/server"
LOG_FILE="/tmp/verifier_startup_pipeline.log"

echo "[CHALLENGE] verifier_startup_pipeline_challenge: START"

# --- Action: Start server and capture logs ---
timeout 60 "${SERVER_BIN}" > "${LOG_FILE}" 2>&1 &
SERVER_PID=$!
sleep 10

# --- Assertions ---

# ANTI-BLUFF 1: Phase 1 (Discover) must log provider discovery
if ! grep -qi "discover" "${LOG_FILE}"; then
    echo "[FAIL] Phase 1 (Discover) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 2: Phase 2 (Verify) must log verification
if ! grep -qi "verif" "${LOG_FILE}"; then
    echo "[FAIL] Phase 2 (Verify) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 3: Phase 3 (Score) must log scoring
if ! grep -qi "score" "${LOG_FILE}"; then
    echo "[FAIL] Phase 3 (Score) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 4: Phase 5 (Debate Team) must select at least 1
# model
if ! grep -qi "debate\\|team\\|selected" "${LOG_FILE}"; then
    echo "[FAIL] Phase 5 (Debate Team) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 5: Server must reach healthy state
if ! curl -sf "http://localhost:8080/health" > /dev/null 2>&1;
then

```

```

    echo "[FAIL] Server health check failed after startup"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

kill "${SERVER_PID}" 2>/dev/null || true
echo "[CHALLENGE] verifier_startup_pipeline_challenge: PASS"

```

Challenge 10: verifier_canned_detection_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "canned response
#             detection works" =
#   a model that returns a canned "I cannot assist" response is
#             marked
#   as NOT verified in the verifier DB.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_canned_detection_challenge: START"

# --- Setup: Insert a model with known canned response pattern
# ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
INSERT OR REPLACE INTO verification_results
(model_id, status, model_exists, responsive,
 supports_code_generation, overall_score,
 code_capability_score)
VALUES (
  (SELECT id FROM models WHERE model_id = 'test-canned-model'),
  'completed',
  1, 1, 0,
  2.0, 1.0
);
UPDATE models SET verification_status = 'failed' WHERE model_id =
'test-canned-model';
EOF

# --- Action: Query model status via API ---
API_OUTPUT="/tmp/verifier_canned_api.json"
curl -sf "http://localhost:8081/api/models/test-canned-model" >
"${API_OUTPUT}" 2>&1 || true

# --- Assertions ---

```

```

# ANTI-BLUFF 1: Model must have verification_status = failed or
    unverified
STATUS=$(python3 -c "
import json, sys
try:
    data = json.load(open('${API_OUTPUT}'))
    print(data.get('verification_status', 'UNKNOWN'))
except:
    print('UNKNOWN')
")

if [[ "${STATUS}" != "failed" && "${STATUS}" != "unverified" ]];
then
    echo "[FAIL] Canned-response model has status '${STATUS}'
        instead of 'failed'"
    exit 1
fi

# ANTI-BLUFF 2: Score must be low (< 3.0)
SCORE=$(python3 -c "
import json, sys
try:
    data = json.load(open('${API_OUTPUT}'))
    print(data.get('overall_score', 999))
except:
    print(999)
")

if (( $(echo "${SCORE} > 3.0" | bc -l) )); then
    echo "[FAIL] Canned-response model has score ${SCORE} (>
        3.0)"
    exit 1
fi

echo "[CHALLENGE] verifier_canned_detection_challenge: PASS"

```

Challenge 11: verifier_security_redaction_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "security redaction
    works" =
# API keys NEVER appear in logs, stdout, stderr, or error
    messages.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

```

```

CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
SERVER_LOG="/tmp/verifier_security_server.log"
CLI_LOG="/tmp/verifier_security_cli.log"

echo "[CHALLENGE] verifier_security_redaction_challenge: START"

# Use a fake but recognizable API key pattern
FAKE_KEY="sk-antibluff-test-key-9876543210abcdef"
export HELIX_OPENAI_API_KEY="${FAKE_KEY}"

# --- Action: Run CLI and capture all output ---
"${CLI_BIN}" --list-models > "${CLI_LOG}" 2>&1 || true

# --- Action: Check server logs ---
if [[ -f "${SERVER_LOG}" ]]; then
    grep -r "${FAKE_KEY}" "${SERVER_LOG}" > /dev/null 2>&1 && {
        echo "[FAIL] API key found in server logs"
        exit 1
    }
fi

# --- Assertions ---

# ANTI-BLUFF 1: Fake key must NOT appear in CLI stdout/stderr
if grep -q "${FAKE_KEY}" "${CLI_LOG}"; then
    echo "[FAIL] API key found in CLI output"
    exit 1
fi

# ANTI-BLUFF 2: Fake key must NOT appear in any log file under
# helix_code/
if grep -r "${FAKE_KEY}" "${PROJECT_ROOT}/helix_code/" > /dev/
null 2>&1; then
    echo "[FAIL] API key found somewhere in HelixCode logs or
output"
    exit 1
fi

# ANTI-BLUFF 3: Config dump must redact keys
CONFIG_OUTPUT="/tmp/verifier_config_dump.txt"
"${CLI_BIN}" --config-dump > "${CONFIG_OUTPUT}" 2>&1 || true

if grep -q "${FAKE_KEY}" "${CONFIG_OUTPUT}"; then
    echo "[FAIL] API key found in config dump output"
    exit 1
fi

```

```
# ANTI-BLUFF 4: Error messages must not contain key fragments
    (first 8 chars)
KEY_FRAGMENT="${FAKE_KEY:0:8}"
if grep -q "${KEY_FRAGMENT}" "${CLI_LOG}"; then
    echo "[FAIL] API key fragment found in CLI output"
    exit 1
fi

echo "[CHALLENGE] verifier_security_redaction_challenge: PASS"
```

Challenge 12: verifier_scoring_accuracy_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "scoring is accurate" =
#   the overall score reflects real verification dimensions,
#   not a hardcoded default like 8.5 for everything.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_scoring_accuracy_challenge: START"

# --- Setup: Insert two models with DIFFERENT scores ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
UPDATE models SET
    overall_score = 9.5,
    code_capability_score = 9.8,
    responsiveness_score = 9.0,
    reliability_score = 9.5,
    feature_richness_score = 9.2,
    value_proposition_score = 8.0
WHERE model_id = 'test-high-score';

UPDATE models SET
    overall_score = 4.0,
    code_capability_score = 3.5,
    responsiveness_score = 5.0,
    reliability_score = 4.0,
    feature_richness_score = 4.5,
    value_proposition_score = 3.0
WHERE model_id = 'test-low-score';
EOF

# --- Action: Query both models via API ---
HIGH_OUTPUT="/tmp/verifier_score_high.json"
LOW_OUTPUT="/tmp/verifier_score_low.json"
```

```

curl -sf "http://localhost:8081/api/models/test-high-score" > "${HIGH_OUTPUT}" 2>&1 || true
curl -sf "http://localhost:8081/api/models/test-low-score" > "${LOW_OUTPUT}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: High-score model must have overall_score >= 9.0
HIGH_SCORE=$(python3 -c "
import json
data = json.load(open('${HIGH_OUTPUT}'))
print(data.get('overall_score', 0))
")
if (( $(echo "${HIGH_SCORE} < 9.0" | bc -l) )); then
    echo "[FAIL] High-score model has score ${HIGH_SCORE} (< 9.0)"
    exit 1
fi

# ANTI-BLUFF 2: Low-score model must have overall_score <= 5.0
LOW_SCORE=$(python3 -c "
import json
data = json.load(open('${LOW_OUTPUT}'))
print(data.get('overall_score', 10))
")
if (( $(echo "${LOW_SCORE} > 5.0" | bc -l) )); then
    echo "[FAIL] Low-score model has score ${LOW_SCORE} (> 5.0)"
    exit 1
fi

# ANTI-BLUFF 3: Scores must be DIFFERENT
if (( $(echo "${HIGH_SCORE} == ${LOW_SCORE}" | bc -l) )); then
    echo "[FAIL] High and low scores are identical (${HIGH_SCORE}) – hardcoded score detected"
    exit 1
fi

# ANTI-BLUFF 4: Score must NOT be exactly 8.5 (the known stub value)
if (( $(echo "${HIGH_SCORE} == 8.5" | bc -l) )) || (( $(echo "${LOW_SCORE} == 8.5" | bc -l) )); then
    echo "[FAIL] Score is exactly 8.5 – likely hardcoded stub value"
    exit 1
fi

echo "[CHALLENGE] verifier_scoring_accuracy_challenge: PASS"

```

7. Security Tests

Scope: API key redaction, secret handling, permissions, encryption.

Mock Policy: NO mocks. Tests real secret handling paths.

Constitutional Basis: CONST-025.

7.1 Security Test File Inventory

#	File Path	Purpose
1	tests/security/ api_key_redaction_test.go	API key redaction in logs and output
2	tests/security/ secrets_in_errors_test.go	No secrets leaked in error messages
3	tests/security/ config_permissions_test.go	Config file permission checks
4	tests/security/ database_encryption_test.go	SQLite encryption verification
5	tests/security/ jwt_secret_handling_test.go	JWT secret isolation
6	tests/security/ provider_key_storage_test.go	Provider key storage security
7	tests/security/ verifier_api_auth_test.go	Verifier API authentication enforcement
8	tests/security/ env_var_exposure_test.go	Environment variable exposure

7.2 Exact Test Function Signatures

File: tests/security/api_key_redaction_test.go

```
func TestAPIKeyRedaction_Logs(t *testing.T)
func TestAPIKeyRedaction_ErrorMessages(t *testing.T)
func TestAPIKeyRedaction_HTTPHeaders(t *testing.T)
func TestAPIKeyRedaction_CLIOutput(t *testing.T)
func TestAPIKeyRedaction_ConfigDump(t *testing.T)
func TestAPIKeyRedaction_DebugEndpoint(t *testing.T)
func TestAPIKeyRedaction_PanicRecovery(t *testing.T)
```

Anti-Bluff Criteria: - TestAPIKeyRedaction_Logs: Must initialize a real logger, perform an operation that would naturally log the API key (e.g., a provider error), capture the log output to a buffer, and assert the string sk- (or the actual key

substring) does NOT appear. A test that only checks a utility function `redact()` is a bluff. - `TestAPIKeyRedaction_PanicRecovery`: Must trigger a real panic inside a function that has the API key in scope, recover, and assert the recovered error/stack does not contain the key.

File: `tests/security/secrets_in_errors_test.go`

```
func TestSecretsNotInErrors_HTTPError(t *testing.T)
func TestSecretsNotInErrors_DBError(t *testing.T)
func TestSecretsNotInErrors_ProviderInitError(t *testing.T)
func TestSecretsNotInErrors_ValidationError(t *testing.T)
func TestSecretsNotInErrors_NetworkError(t *testing.T)
func TestSecretsNotInErrors_JSONMarshalError(t *testing.T)
```

Anti-Bluff Criteria: - Each test must use a REAL API key (test key), trigger the specific error condition, capture the error string, and assert the key is not present. A test using `errors.New("some error")` and checking `redact()` is a bluff.

File: `tests/security/config_permissions_test.go`

```
func TestConfigPermissions_CreationMode(t *testing.T)
func TestConfigPermissions_WorldReadable(t *testing.T)
func TestConfigPermissions_WorldWritable(t *testing.T)
func TestConfigPermissions_SecretFile(t *testing.T)
```

Anti-Bluff Criteria: - `TestConfigPermissions_WorldReadable`: Must create a config file with an embedded API key, set mode `0644`, then assert the application refuses to load it or logs a security warning. If the app loads it without complaint, test FAILs. - `TestConfigPermissions_SecretFile`: Must create a `.env` file with mode `0600`, assert it is accepted. Then change to `0644` and assert rejection or warning.

File: `tests/security/database_encryption_test.go`

```
func TestDatabaseEncryption_SQLCipherEnabled(t *testing.T)
func TestDatabaseEncryption_SQLCipherDisabled(t *testing.T)
func TestDatabaseEncryption_KeyRotation(t *testing.T)
func TestDatabaseEncryption_PlaintextNotInFile(t *testing.T)
```

Anti-Bluff Criteria: - `TestDatabaseEncryption_PlaintextNotInFile`: Must create a verifier DB with encryption ON, insert a model with a unique name (e.g., `ENCRYPTION_TEST_12345`), then read the raw SQLite file bytes and assert the string `ENCRYPTION_TEST_12345` does NOT appear. Finding the plaintext string causes failure.

8. Performance Tests

Scope: Latency, memory, concurrency, throughput. **Mock Policy:** NO mocks. Real verifier, real cache, real DB. **Constitutional Basis:** CONST-014.

8.1 Performance Test File Inventory

#	File Path	Purpose
1	tests/performance/ model_list_latency_test.go	Model list retrieval latency
2	tests/performance/ verifier_polling_overhead_test.go	Verifier polling CPU overhead
3	tests/performance/ memory_model_registry_test.go	Memory with 100+ models
4	tests/performance/ concurrent_registry_test.go	Concurrent model registry access
5	tests/performance/ scoring_latency_test.go	Scoring calculation latency
6	tests/performance/ discovery_latency_test.go	Discovery latency with many providers
7	tests/performance/ cache_hit_latency_test.go	Cache hit latency
8	tests/performance/ startup_pipeline_latency_test.go	Full startup pipeline latency

8.2 Exact Test Function Signatures

File: tests/performance/model_list_latency_test.go

```
func BenchmarkModelList_Cached(b *testing.B)
func BenchmarkModelList_Uncached(b *testing.B)
func TestModelListLatency_Cached_Under500ms(t *testing.T)
func TestModelListLatency_Uncached_Under2s(t *testing.T)
func TestModelListLatency_FirstCall_Under5s(t *testing.T)
```

Anti-Bluff Criteria: - TestModelListLatency_Cached_Under500ms: Must measure WALL CLOCK time (not CPU time) for 100 sequential calls and assert the 95th percentile is under 500ms. A test that measures a single call and asserts < 500ms is insufficient — it must be statistically meaningful. - The cache must be primed before measurement. If the cache is cold, the test must FAIL.

File: tests/performance/memory_model_registry_test.go

```
func TestMemory_100Models(t *testing.T)
func TestMemory_500Models(t *testing.T)
func TestMemory_1000Models(t *testing.T)
func TestMemory_ModelRegistry_GCStable(t *testing.T)
func TestMemory_ModelRegistry_NoLeak(t *testing.T)
```

Anti-Bluff Criteria: - TestMemory_100Models: Must register 100 models, force GC, and assert RSS increase is under 50MB. Must print the actual memory delta. A test without runtime.GC() and runtime.ReadMemStats() is a bluff. - TestMemory_ModelRegistry_NoLeak: Must register and unregister models in a loop (1000 iterations), force GC, and assert final memory is within 10% of baseline. Growing memory without bound causes failure.

File: tests/performance/concurrent_registry_test.go

```
func TestConcurrentRegistry_ReadWrite(t *testing.T)
func TestConcurrentRegistry_MultipleReaders(t *testing.T)
func TestConcurrentRegistry_WriteDuringRead(t *testing.T)
func TestConcurrentRegistry_RaceDetection(t *testing.T)
```

Anti-Bluff Criteria: - TestConcurrentRegistry_RaceDetection: Must run with go test -race and assert NO race conditions are detected. A test that runs without -race is a bluff. - TestConcurrentRegistry_ReadWrite: Must use 100 goroutines (50 readers, 50 writers) for 5 seconds and assert no panics, no deadlocks, and correct final state.

File: tests/performance/verifier_polling_overhead_test.go

```
func TestVerifierPollingOverhead_CPUUnder5Percent(t *testing.T)
func TestVerifierPollingOverhead_NoSpikes(t *testing.T)
```

Anti-Bluff Criteria: - TestVerifierPollingOverhead_CPUUnder5Percent: Must start the verifier polling loop, measure CPU usage for 30 seconds, and assert average CPU is under 5%. A test without actual CPU measurement is a bluff.

9. Coverage Enforcement

9.1 Coverage Target: 100%

Per **CONST-002**, the target is **100% coverage across all supported test types**. This means:

Test Type	Coverage Metric	Enforcement
Unit	Line coverage	100% of lines in internal/verifier/, internal/llm/ verifier_*.go, cmd/cli/verifier_*.go
Contract	API endpoint coverage	100% of documented endpoints exercised
Component	Subsystem interaction coverage	100% of subsystem pairs wired and tested
Integration	Dependency coverage	100% of configured dependencies (PG, Redis, SQLite, providers) exercised
E2E / Challenge	Feature coverage	100% of user-facing features have a challenge script
Security	Attack surface coverage	100% of secret-handling paths tested
Performance	Benchmark coverage	Every public function with >10ms expected latency has a benchmark

9.2 Coverage Measurement Mechanism

Go Line Coverage

Unit + Component coverage

```
go test -coverprofile=coverage-unit.out -short ./internal/
verifier/... ./internal/llm/... ./cmd/cli/...
```

Integration coverage (excludes unit-only files)

```
go test -coverprofile=coverage-integration.out -run
Integration ./tests/integration/...
```

Combined coverage

```
go tool cover -func=coverage-unit.out | tail -1
go tool cover -func=coverage-integration.out | tail -1
```

Coverage Enforcement Script

File: scripts/enforce_coverage.sh

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
UNIT_THRESHOLD=100
```

```
INTEGRATION_THRESHOLD=95
```

```
CONTRACT_THRESHOLD=100
```

```
SECURITY_THRESHOLD=95
```

```
PERFORMANCE_THRESHOLD=80
```

```
# Unit coverage
```

```
UNIT_COVER=$(go test -short ./internal/verifier/... ./internal/  
    llm/... ./cmd/cli/... -cover 2>/dev/null | grep -oP '\d+  
    \.\d+%' | tail -1 | tr -d '%')
```

```
if (( $(echo "${UNIT_COVER} < ${UNIT_THRESHOLD}" | bc -l) ));
```

```
then
```

```
    echo "[FAIL] Unit coverage ${UNIT_COVER}% < ${UNIT_THRESHOLD}  
    %"
```

```
    exit 1
```

```
fi
```

```
# Integration coverage
```

```
INTEGRATION_COVER=$(go test -run Integration ./tests/  
    integration/... -cover 2>/dev/null | grep -oP '\d+\.\d+  
    %' | tail -1 | tr -d '%')
```

```
if (( $(echo "${INTEGRATION_COVER} < ${INTEGRATION_THRESHOLD}" |  
    bc -l) )); then
```

```
    echo "[FAIL] Integration coverage ${INTEGRATION_COVER}% < $  
    {INTEGRATION_THRESHOLD}%"
```

```
    exit 1
```

```
fi
```

```
echo "[PASS] Coverage check passed"
```

9.3 No-Mocks-Above-Unit Scanner

File: scripts/no_mock_above_unit.sh

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
# CONST-021: No Mocks Above Unit
```

```
# Scans all test files outside of *_test.go (short mode) for mock  
    usage
```

VIOLATIONS=0

```
# Find all test files that are NOT in unit-test-only directories
for FILE in $(find tests/ internal/ -name "*_test.go" | grep -v
    "^internal/verifier/*_test.go" | grep -v "^internal/
    llm/*_test.go" | grep -v "^cmd/cli/*_test.go"); do
    if grep -qE "(mock|Mock|gomock|mockery|httptest)" "${FILE}";
    then
        echo "[VIOLATION] Mock usage found in non-unit test: $
        {FILE}"
        VIOLATIONS=$((VIOLATIONS + 1))
    fi
done

if [[ "${VIOLATIONS}" -gt 0 ]]; then
    echo "[FAIL] ${VIOLATIONS} mock violations found above unit
    test level"
    exit 1
fi

echo "[PASS] No mocks above unit tests"
```

Note: httptest is allowed in unit tests for local HTTP server simulation. It is NOT allowed in integration or component tests because those must use real running servers.

10. Test Infrastructure

10.1 Docker Compose for Test Dependencies

File: docker/docker-compose.test.yml

```
version: "3.9"

services:
  postgres-test:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: helix
      POSTGRES_PASSWORD: ${HELIX_DATABASE_PASSWORD:?set
        HELIX_DATABASE_PASSWORD in .env (run ./setup.sh, or
        cp .env.example .env)}
      POSTGRES_DB: helixcode_test
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U helix"]
```

```
    interval: 2s
    timeout: 5s
    retries: 10
tmpfs:
  - /var/lib/postgresql/data
```

```
redis-test:
  image: redis:7-alpine
  ports:
    - "6380:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 2s
    timeout: 5s
    retries: 10
```

```
verifier-test:
  build:
    context: ../LLMsVerifier
    dockerfile: Dockerfile
  environment:
    - VERIFIER_DATABASE_PATH=/data/verifier-test.db
    - VERIFIER_API_PORT=8081
  ports:
    - "8081:8081"
  volumes:
    - verifier-test-data:/data
  healthcheck:
    test: ["CMD-SHELL", "curl -sf http://localhost:8081/api/health || exit 1"]
    interval: 5s
    timeout: 5s
    retries: 10
```

```
ollama-test:
  image: ollama/ollama:latest
  ports:
    - "11435:11434"
  # Pull a tiny model for testing
  entrypoint: >
    sh -c "ollama serve &
    sleep 5 &&
    ollama pull llama3.2:1b &&
    wait"
```

```
volumes:
  verifier-test-data:
```

10.2 Test Data Fixtures

Directory: tests/fixtures/

File	Purpose
fixtures/ verifier_db_seed.sql	SQLite seed data for verifier DB
fixtures/ provider_responses/	Cached real provider API responses (for offline contract validation)
fixtures/configs/	Test configuration files (YAML/JSON/TOML)
fixtures/keys/	Test API keys (dummy values, NOT real)
fixtures/models/	Test model metadata JSON files

File: tests/fixtures/verifier_db_seed.sql

```
-- Seed data for verifier test database
INSERT INTO providers (name, type, base_url, status) VALUES
('openai', 'openai', 'https://api.openai.com/v1', 'active'),
('anthropic', 'anthropic', 'https://api.anthropic.com/v1',
    'active'),
('deepseek', 'deepseek', 'https://api.deepseek.com/v1',
    'active');

INSERT INTO models (provider_id, model_id, name, description,
    overall_score, verification_status)
VALUES
(1, 'gpt-4o', 'GPT-4o', 'OpenAI GPT-4o', 9.2, 'verified'),
(1, 'gpt-4o-mini', 'GPT-4o Mini', 'OpenAI GPT-4o Mini', 8.5,
    'verified'),
(2, 'claude-sonnet-4', 'Claude Sonnet 4', 'Anthropic Claude
    Sonnet', 8.8, 'verified'),
(3, 'deepseek-chat', 'DeepSeek Chat', 'DeepSeek V3', 8.5,
    'verified');
```

10.3 Environment Setup Scripts

File: scripts/setup_test_env.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Setup test environment for LLMsVerifier integration tests

echo "[SETUP] Starting test environment setup..."
```

```

# 1. Build CLI and server binaries
cd "${PROJECT_ROOT}/HelixCode"
make build-cli build-server

# 2. Start Docker Compose test infrastructure
docker compose -f "${PROJECT_ROOT}/docker/docker-
compose.test.yml" up -d --wait

# 3. Seed verifier test database
docker compose -f "${PROJECT_ROOT}/docker/docker-
compose.test.yml" exec -T verifier-test \
sqlite3 /data/verifier-test.db < "${PROJECT_ROOT}/tests/
fixtures/verifier_db_seed.sql"

# 4. Verify all services are healthy
for SERVICE in postgres-test redis-test verifier-test; do
    if ! docker compose -f "${PROJECT_ROOT}/docker/docker-
compose.test.yml" ps "${SERVICE}" | grep -q "healthy";
    then
        echo "[FAIL] Service ${SERVICE} is not healthy"
        exit 1
    fi
done

# 5. Export test environment variables
export HELIX_DATABASE_HOST=localhost
export HELIX_DATABASE_PORT=5433
export HELIX_DATABASE_PASSWORD=CHANGE_ME_db_password
export HELIX_REDIS_HOST=localhost
export HELIX_REDIS_PORT=6380
export HELIX_VERIFIER_URL=http://localhost:8081

echo "[SETUP] Test environment ready"

```

File: scripts/teardown_test_env.sh

```

#!/usr/bin/env bash
set -euo pipefail

echo "[TEARDOWN] Stopping test environment..."
docker compose -f "${PROJECT_ROOT}/docker/docker-
compose.test.yml" down -v
echo "[TEARDOWN] Test environment stopped"

```

10.4 Test Runner Script

File: scripts/run_tests.sh

```

#!/usr/bin/env bash
set -euo pipefail

TEST_TYPE="${1:-all}"

case "${TEST_TYPE}" in
    unit)
        echo "[TEST] Running unit tests..."
        go test -short -v ./internal/verifier/... ./internal/
            llm/... ./cmd/cli/... ./internal/services/...
        ;;
    contract)
        echo "[TEST] Running contract tests..."
        go test -v -run Contract ./tests/contract/...
        ;;
    component)
        echo "[TEST] Running component tests..."
        go test -v -run Component ./tests/component/...
        ;;
    integration)
        echo "[TEST] Running integration tests..."
        go test -v -run Integration ./tests/integration/...
        ;;
    e2e|challenge)
        echo "[TEST] Running E2E challenge scripts..."
        cd "${PROJECT_ROOT}/challenges/scripts"
        for SCRIPT in verifier*_challenge.sh; do
            echo "[TEST] Running ${SCRIPT}..."
            bash "${SCRIPT}"
        done
        ;;
    security)
        echo "[TEST] Running security tests..."
        go test -v -run Security ./tests/security/...
        ;;
    performance)
        echo "[TEST] Running performance tests..."
        go test -v -bench=. -benchmem ./tests/performance/...
        ;;
    coverage)
        echo "[TEST] Running coverage enforcement..."
        bash "${PROJECT_ROOT}/scripts/enforce_coverage.sh"
        bash "${PROJECT_ROOT}/scripts/no_mock_above_unit.sh"
        ;;
    all|complete)
        echo "[TEST] Running complete test suite..."
        bash "${PROJECT_ROOT}/scripts/setup_test_env.sh"
        make test-unit-full

```

```

make test-contract-full
make test-component-full
make test-integration-full
make test-e2e-full
make test-security-full
make test-load-full
make coverage-full
bash "${PROJECT_ROOT}/scripts/teardown_test_env.sh"
;;
*)
echo "Unknown test type: ${TEST_TYPE}"
echo "Usage: $0 {unit|contract|component|integration|e2e|
security|performance|coverage|all}"
exit 1
;;
esac

```

11. Anti-Bluff Verification Checklist Matrix

This matrix maps every user-facing feature to the test that proves it works and the exact verification method.

Feature	Test File / Challenge
Model listing (CLI)	verifier_model_list_challenge.sh
Model listing (API)	tests/integration/ helixcode_full_stack_test.go:TestFullStack_APIModels_ReturnsV
Model selection with scoring	tests/component/ model_manager_verifier_component_test.go:TestModelManager_Se

Feature	Test File / Challenge
Code generation	verifier_model_select_challenge.sh
Verifier disable + fallback	verifier_disable_fallback_challenge.sh
API key provisioning	verifier_api_key_provision_challenge.sh
Rate limiting display	verifier_rate_limit_display_challenge.sh
Real-time updates	verifier_realtime_update_challenge.sh
MCP integration	verifier_mcp_lsp_acp_challenge.sh
LSP integration	verifier_mcp_lsp_acp_challenge.sh

Feature	Test File / Challenge
ACP integration	verifier_mcp_lsp_acp_challenge.sh
Embedding integration	verifier_mcp_lsp_acp_challenge.sh
Cross-platform CLI	verifier_cross_platform_cli_challenge.sh
Startup pipeline	verifier_startup_pipeline_challenge.sh
Canned response detection	verifier_canned_detection_challenge.sh
Security redaction	verifier_security_redaction_challenge.sh
Scoring accuracy	verifier_scoring_accuracy_challenge.sh
Cache hit latency	tests/performance/model_list_latency_test.go

Feature	Test File / Challenge
Database encryption	tests/security/database_encryption_test.go
Provider fallback chain	tests/integration/helixcode_fallback_chain_test.go
Health monitoring	tests/component/startup_pipeline_component_test.go
Config hot-reload	tests/integration/helixcode_verifier_config_reload_test.go
Event publishing	tests/component/event_bus_verifier_component_test.go
Alias resolution	internal/verifier/aliases_test.go
Subscription detection	internal/verifier/subscription_detector_test.go

Feature	Test File / Challenge
Score suffix format	internal/verifier/scoring_test.go
Verification result persistence	tests/integration/helixcode_verifier_sqlite_test.go
Rate limit header parsing	internal/verifier/rate_limit_test.go
Concurrent model registry	tests/performance/concurrent_registry_test.go
Memory stability	tests/performance/memory_model_registry_test.go
API schema validation	tests/contract/schema_validation_test.go
Error response format	tests/contract/error_response_contract_test.go
JWT auth on verifier API	tests/security/verifier_api_auth_test.go

Feature	Test File / Challenge
Config file permissions	tests/security/config_permissions_test.go

Appendix A: Makefile Target Definitions

Add these targets to helix_code/Makefile:

```
# --- Test Infrastructure ---
.PHONY: test-infra-up test-infra-down test-infra-status
test-infra-up:
    docker compose -f ../docker/docker-compose.test.yml up -d --
    wait

test-infra-down:
    docker compose -f ../docker/docker-compose.test.yml down -v

test-infra-status:
    docker compose -f ../docker/docker-compose.test.yml ps

# --- Unit Tests ---
.PHONY: test-unit test-unit-full test-unit-coverage
test-unit:
    go test -short -v ./internal/verifier/... ./internal/
    llm/... ./cmd/cli/... ./internal/services/...

test-unit-full: test-infra-up
    go test -short -v -race -coverprofile=coverage-unit.out ./
    internal/verifier/... ./internal/llm/... ./cmd/
    cli/... ./internal/services/...
    go tool cover -func=coverage-unit.out | tail -1

test-unit-coverage:
    go test -short -coverprofile=coverage-unit.out ./internal/
    verifier/... ./internal/llm/... ./cmd/cli/... ./
    internal/services/...
    go tool cover -html=coverage-unit.out -o coverage-unit.html

# --- Contract Tests ---
```

```
.PHONY: test-contract test-contract-full
test-contract: test-infra-up
    go test -v -run Contract ./tests/contract/...

test-contract-full: test-infra-up
    go test -v -race -run Contract -coverprofile=coverage-
        contract.out ./tests/contract/...

# --- Component Tests ---
.PHONY: test-component test-component-full
test-component: test-infra-up
    go test -v -run Component ./tests/component/...

test-component-full: test-infra-up
    go test -v -race -run Component -coverprofile=coverage-
        component.out ./tests/component/...

# --- Integration Tests ---
.PHONY: test-integration test-integration-full
test-integration: test-infra-up
    go test -v -run Integration ./tests/integration/...

test-integration-full: test-infra-up
    go test -v -race -run Integration -coverprofile=coverage-
        integration.out ./tests/integration/...

# --- E2E / Challenge Tests ---
.PHONY: test-e2e test-e2e-full
test-e2e:
    cd ../challenges/scripts && bash
        run_all_verifier_challenges.sh

test-e2e-full: test-infra-up build-cli build-server
    cd ../challenges/scripts && bash
        run_all_verifier_challenges.sh

# --- Security Tests ---
.PHONY: test-security test-security-full
test-security: test-infra-up
    go test -v -run Security ./tests/security/...

test-security-full: test-infra-up
    go test -v -race -run Security -coverprofile=coverage-
        security.out ./tests/security/...

# --- Performance / Load Tests ---
.PHONY: test-performance test-load-full
test-performance: test-infra-up
```

```

go test -v -bench=. -benchmem -benchtime=30s ./tests/
performance/...

test-load-full: test-infra-up
go test -v -bench=. -benchmem -benchtime=60s
-cpuprofile=cpu.prof -memprofile=mem.prof ./tests/
performance/...

# --- Coverage & Quality Gates ---
.PHONY: coverage-full no-mocks-above-unit
coverage-full: test-unit-full test-contract-full test-component-
full test-integration-full
@echo "--- Combined Coverage Report ---"
go tool cover -func=coverage-unit.out | tail -1
go tool cover -func=coverage-contract.out | tail -1
go tool cover -func=coverage-component.out | tail -1
go tool cover -func=coverage-integration.out | tail -1

no-mocks-above-unit:
bash ../scripts/no_mocks_above_unit.sh

# --- Complete Test Suite ---
.PHONY: test-complete test-full
test-complete: test-unit-full test-contract-full test-component-
full test-integration-full test-e2e-full test-security-
full test-load-full coverage-full no-mocks-above-unit
@echo "====="
@echo "ALL TESTS COMPLETE"
@echo "====="

test-full: test-complete

# --- Documentation Tests ---
.PHONY: test-docs
test-docs:
# Verify all documented test files exist
@test -f internal/verifier/client_test.go || (echo "MISSING:
client_test.go"; exit 1)
@test -f tests/contract/verifier_api_contract_test.go ||
(echo "MISSING: verifier_api_contract_test.go"; exit 1)
@test -f tests/component/
verifier_client_cache_component_test.go || (echo
"MISSING:
verifier_client_cache_component_test.go"; exit 1)
@test -f tests/integration/helixcode_verifier_sqlite_test.go
|| (echo "MISSING: helixcode_verifier_sqlite_test.go";
exit 1)

```

```

@test -f tests/security/api_key_redaction_test.go || (echo
    "MISSING: api_key_redaction_test.go"; exit 1)
@test -f tests/performance/model_list_latency_test.go ||
    (echo "MISSING: model_list_latency_test.go"; exit 1)
@test -f challenges/scripts/verifier_model_list_challenge.sh
    || (echo "MISSING: verifier_model_list_challenge.sh";
        exit 1)
@echo "[PASS] All documented test files exist"

```

Appendix B: Constitution Cross-Reference

Constitution ID	Requirement	Test Strategy Implementation
CONST-001	No CI/CD	All tests run via Makefile targets and shell scripts; no GitHub Actions, no Jenkins, no pipeline YAML
CONST-002	100% Test Coverage	scripts/enforce_coverage.sh enforces 100% unit, 95%+ integration; coverage-full target generates reports
CONST-002a	No Mocks Above Unit	scripts/no_mock_above_unit.sh scans and rejects any mock usage outside *_test.go short-mode files
CONST-003	No HTTPS for Git	N/A for testing (SSH-only repo access enforced elsewhere)
CONST-004	No manual container commands	docker-compose.test.yml is declarative; make test-infra-up is the only orchestrator-approved entry point
CONST-005	100% real data for non-unit tests	All contract/component/integration/e2e/security tests use real DBs, real APIs, real keys, real CLI binary
CONST-006	Challenge coverage for every component	12 challenge scripts cover every verifier component; matrix in Section 11 maps components to challenges
CONST-017	Zero-Bluff Testing	This entire document is the implementation; every test has anti-bluff criteria
CONST-020		

Constitution ID	Requirement	Test Strategy Implementation
	Provider Fallback Chain Reality	helixcode_fallback_chain_test.go tests with real wrong endpoints, not mock errors
CONST-021	No Mocks Above Unit target	make no-mocks-above-unit runs the scanner
CONST-025	Secret Management	Security tests verify keys are never in logs/errors/output; config permissions enforce 0600
CONST-035	End-User Usability Mandate	Every challenge script verifies exact user-visible output; t.Skip() requires SKIP-OK justification

Summary of Deliverables

Category	Count	Files
New Unit Test Files	20	internal/verifier/*_test.go, internal/llm/verifier*_test.go, cmd/cli/verifier_cli_test.go, internal/services/ llmsverifier_score_adapter_test.go
New Contract Test Files	6	tests/contract/*_contract_test.go
New Component Test Files	6	tests/component/ *_component_test.go
New Integration Test Files	8	tests/integration/ helixcode*_test.go
New Challenge Scripts	12	challenges/scripts/ verifier*_challenge.sh
New Security Test Files	8	tests/security/*_test.go
New Performance Test Files	8	tests/performance/*_test.go
Docker Compose	1	docker/docker-compose.test.yml
Test Fixtures	5+	tests/fixtures/*
	3	

Category	Count	Files
Setup/Teardown Scripts		scripts/setup_test_env.sh, scripts/teardown_test_env.sh, scripts/run_tests.sh
Coverage Enforcement	2	scripts/enforce_coverage.sh, scripts/no_mock_above_unit.sh
Makefile Targets	20+	Added to helix_code/Makefile

Total New Files: ~75

Total Estimated Lines: ~12,000+

Constitutional Compliance: 100% (CONST-001 through CONST-035)

End of Anti-Bluff Testing Strategy for LLMsVerifier Integration into HelixCode